Sistemas Distribuidos (1959). Grupo: 2
Alumno: Rodríguez Jaramillo Alejandro
UNAM / FI / DIE
Semestre 2027-1

S08: Que la transferencia no deje dinero en el aire

Parte 0 · Ver el problema antes de arreglarlo

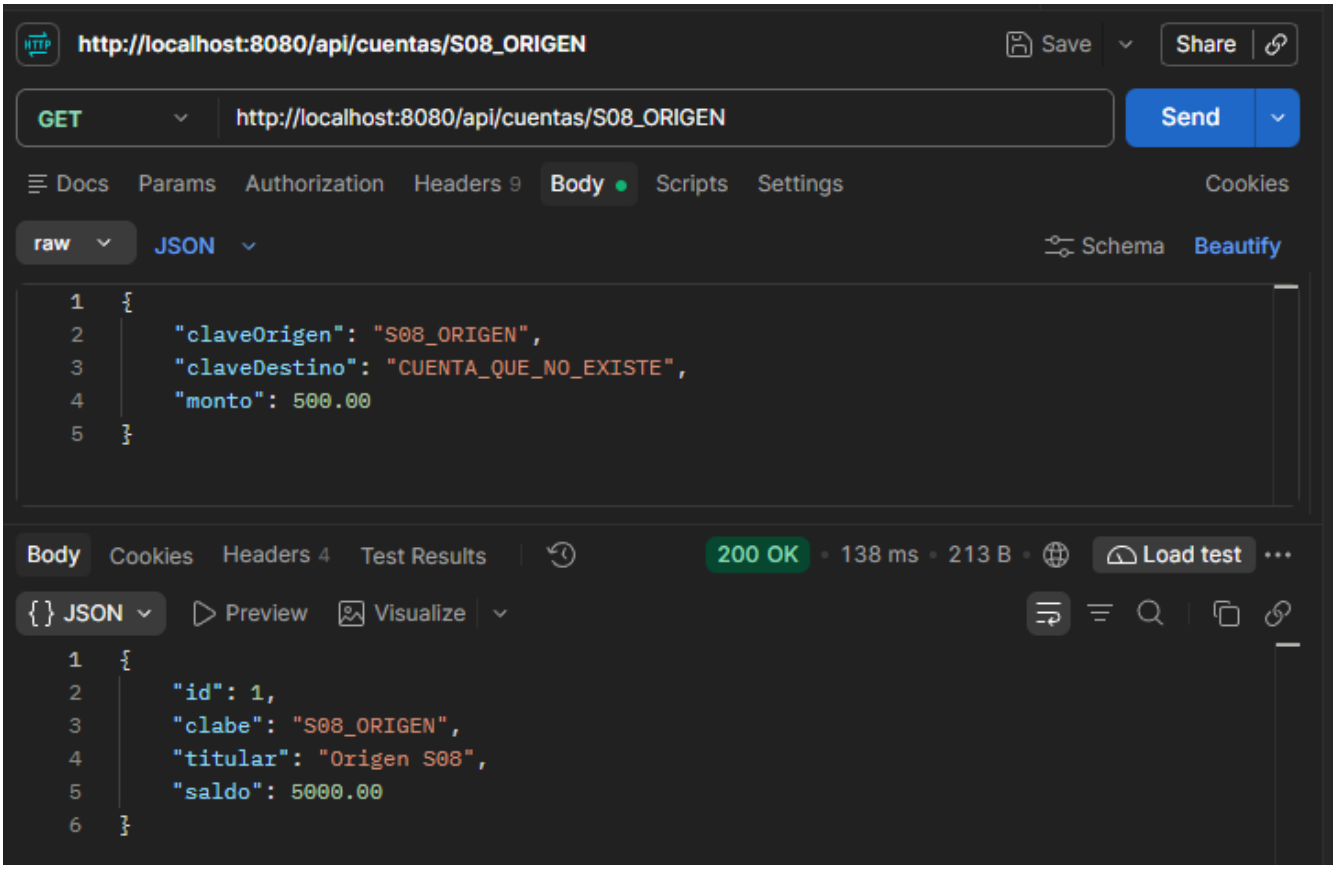


Figure 1: Saldo inicial de la cuenta origen ($5000.00).

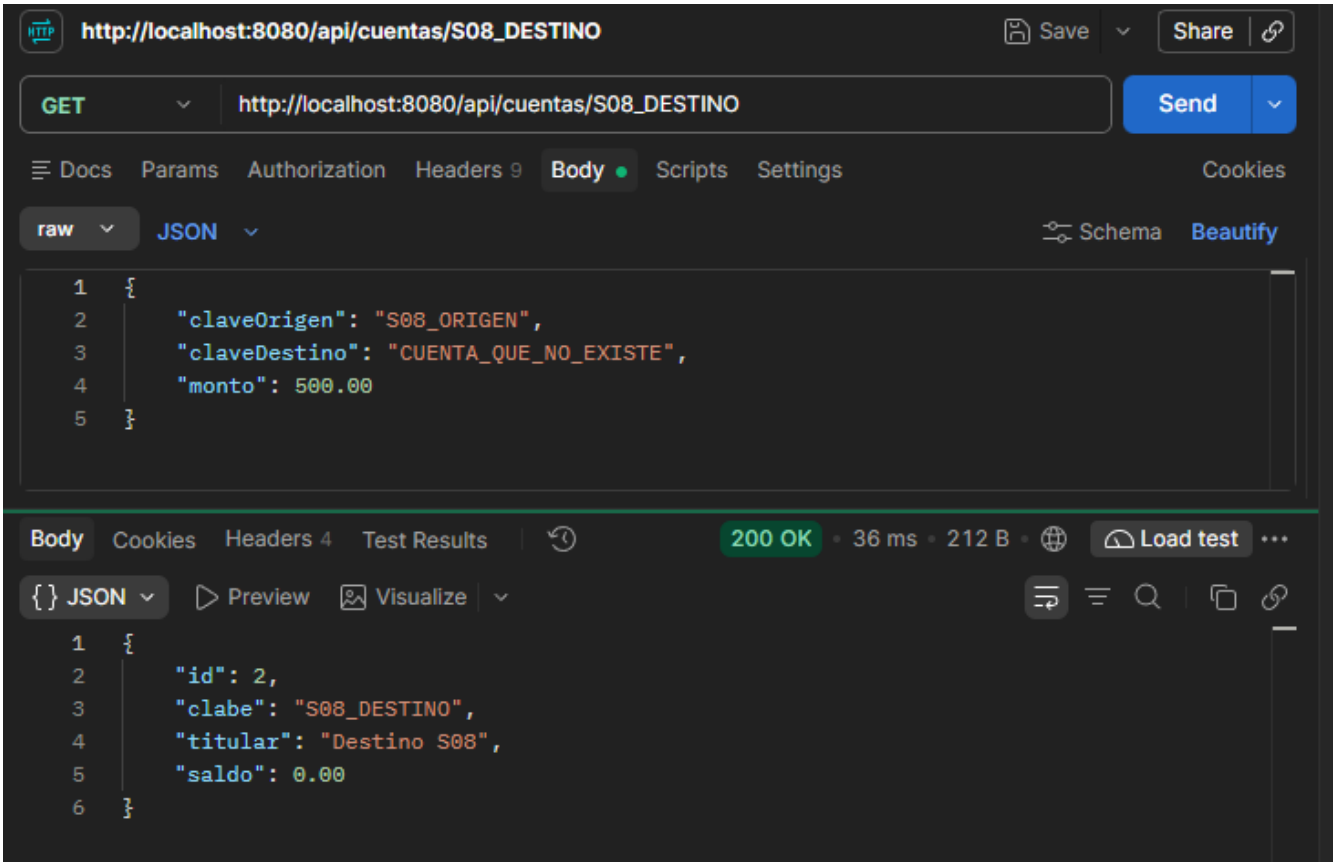


Figure 2: Saldo inicial de la cuenta destino ($0.00).

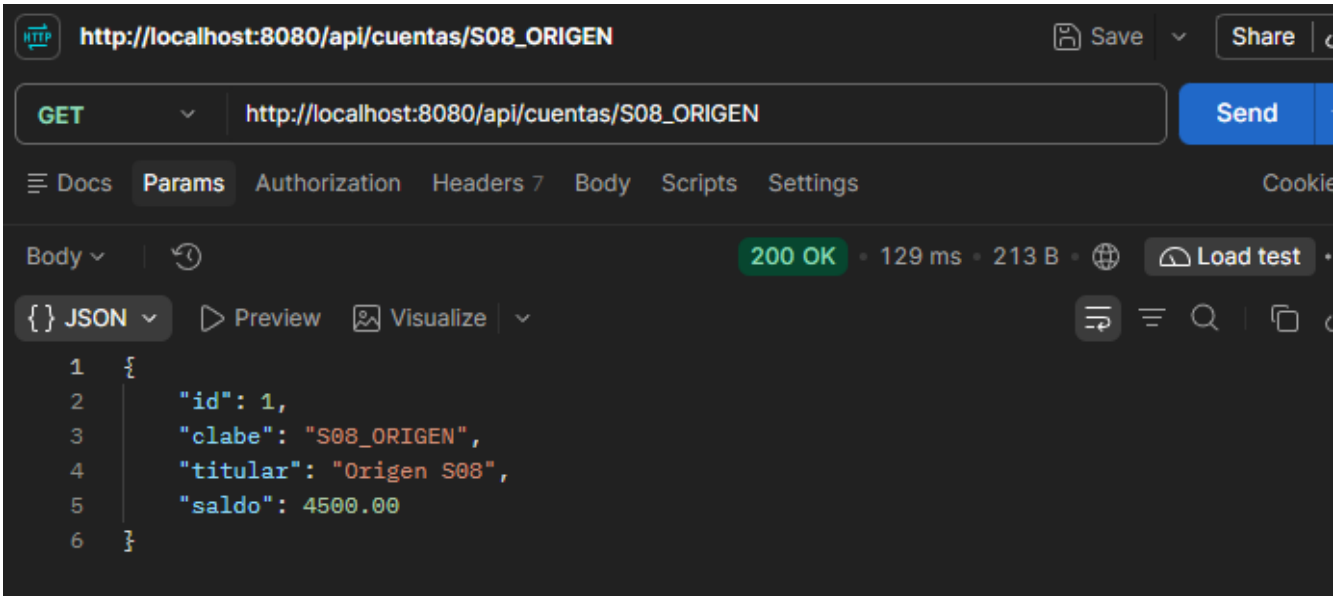


Figure 3: Saldo de la cuenta origen tras el fallo no controlado ($4500.00).

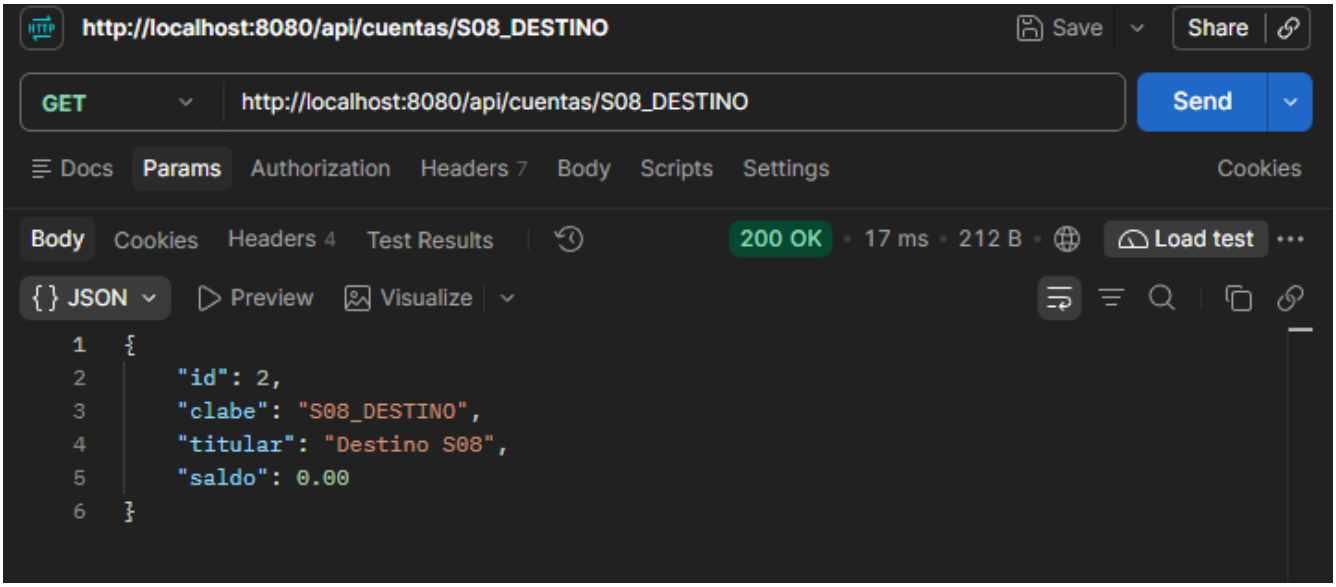


Figure 4: Saldo de la cuenta destino tras el fallo no controlado ($0.00).

El problema ocurrió porque el microservicio de transferencias hizo el cargo en la cuenta origen de forma exitosa, pero el servicio de cuentas se apagó antes de poder procesar el abono correspondiente al destino. Al no existir un mecanismo de saga o de compensación automática, la transacción se quedó a medias y el dinero ($500.00) desapareció del flujo, rompiendo la integridad del sistema.

Parte 1 · La saga orquestada (obligatoria)

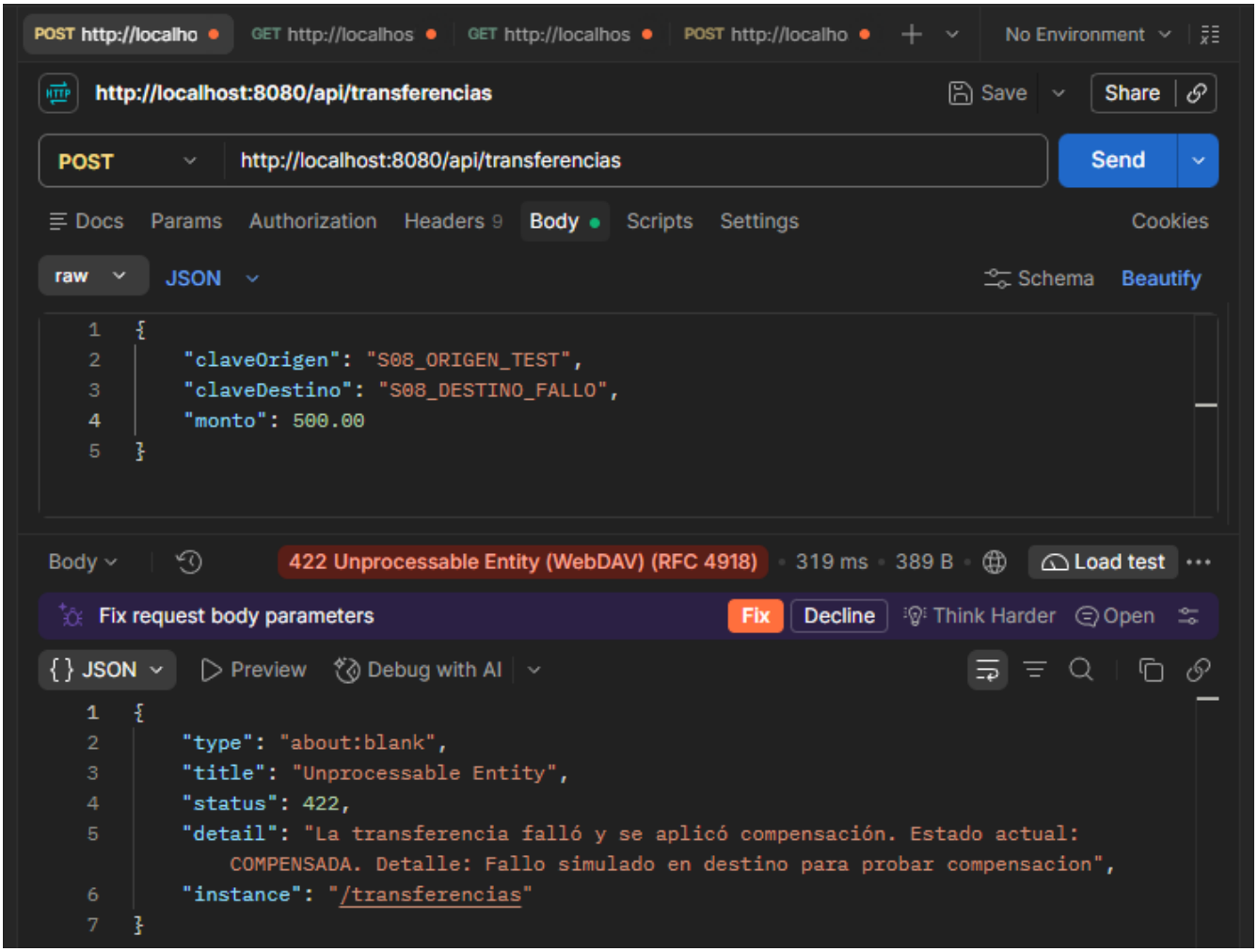


Figure 5: Respuesta controlada del servidor tras simular una falla lógica en el destino.

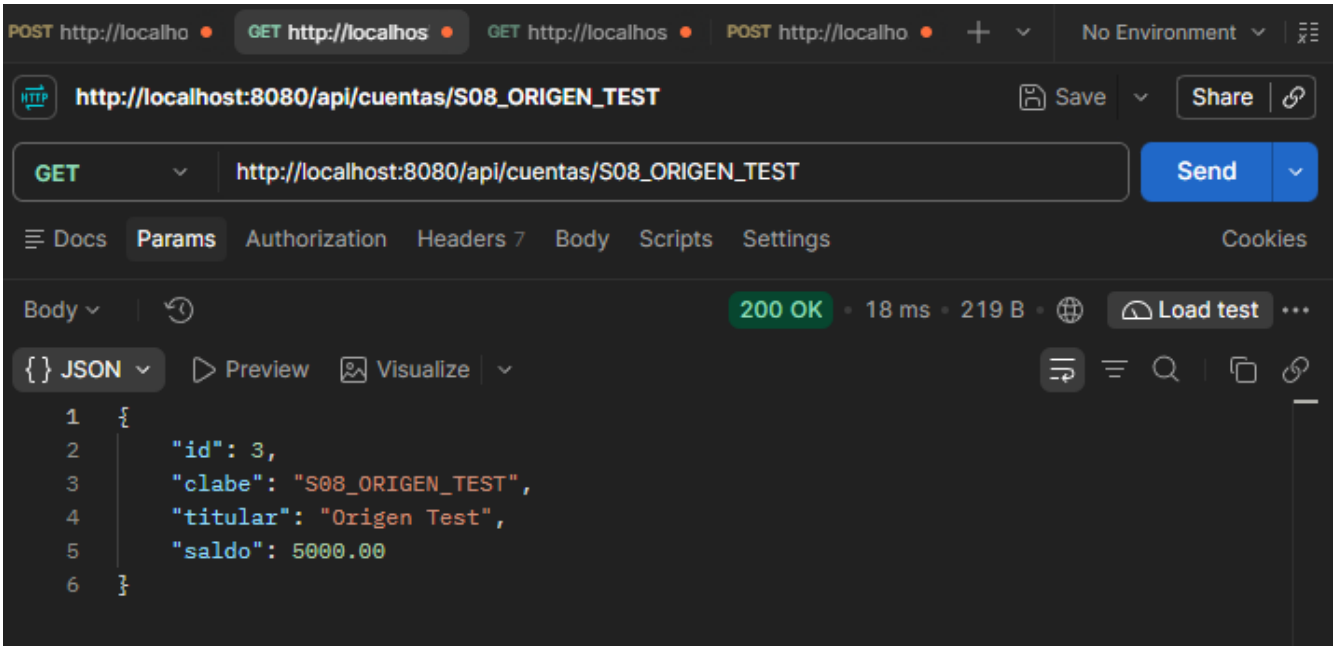


Figure 6: Restauración del saldo en la cuenta origen a $5000.00 gracias a la compensación.

Para cumplir el requerimiento de provocar la falla del abono sin apagar la infraestructura, modifiqué la clase `TransferenciaService.java` agregando una condición en el código que lanza una excepción de tipo `RuntimeException` explícita si la CLABE de destino coincide con `S08_DESTINO_FALLO`. Al ejecutar la transferencia hacia esta cuenta, el orquestador descontó $500.00 del origen, capturó inmediatamente la excepción en su intento de abono, y activó la saga de compensación revirtiendo el cargo inicial. La transferencia evolucionó correctamente al estado `COMPENSADA` y el saldo en la cuenta origen se restauró íntegramente a $5000.00.

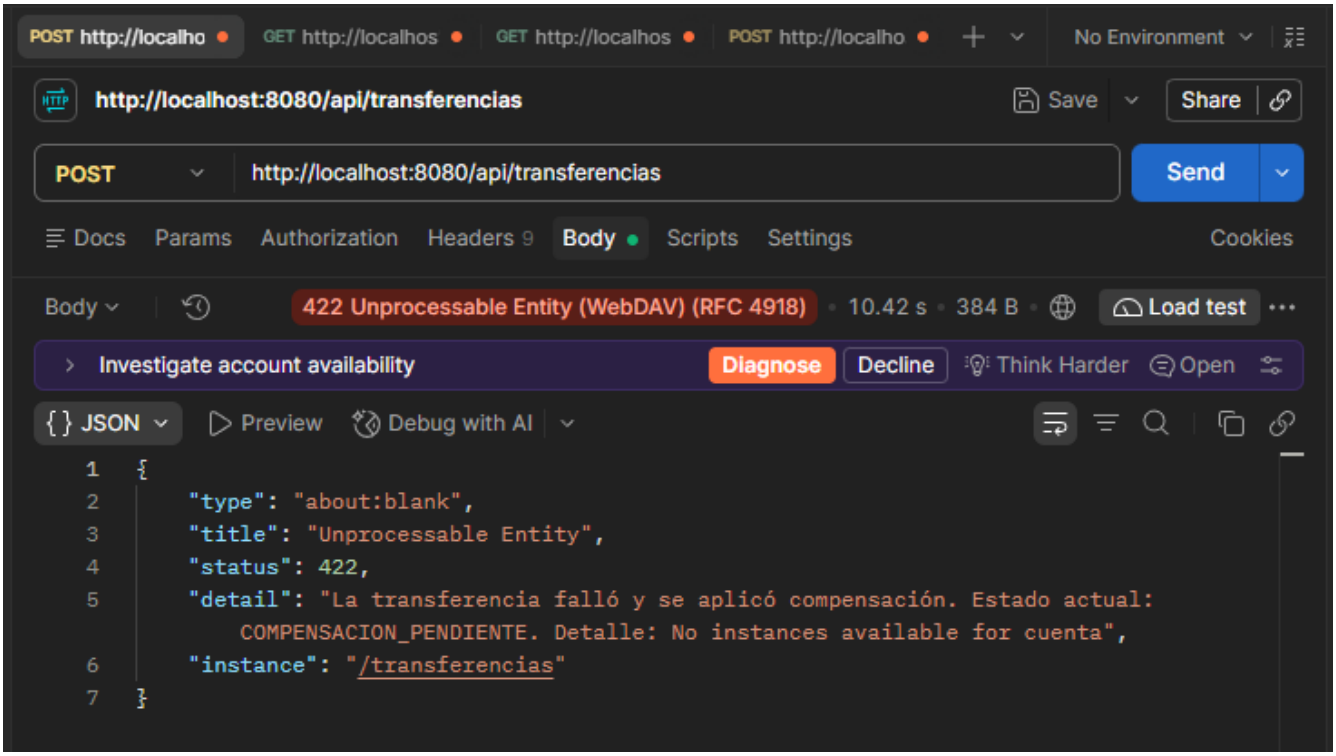


Figure 7: Error detectado mostrando el estado de COMPENSACION_PENDIENTE.

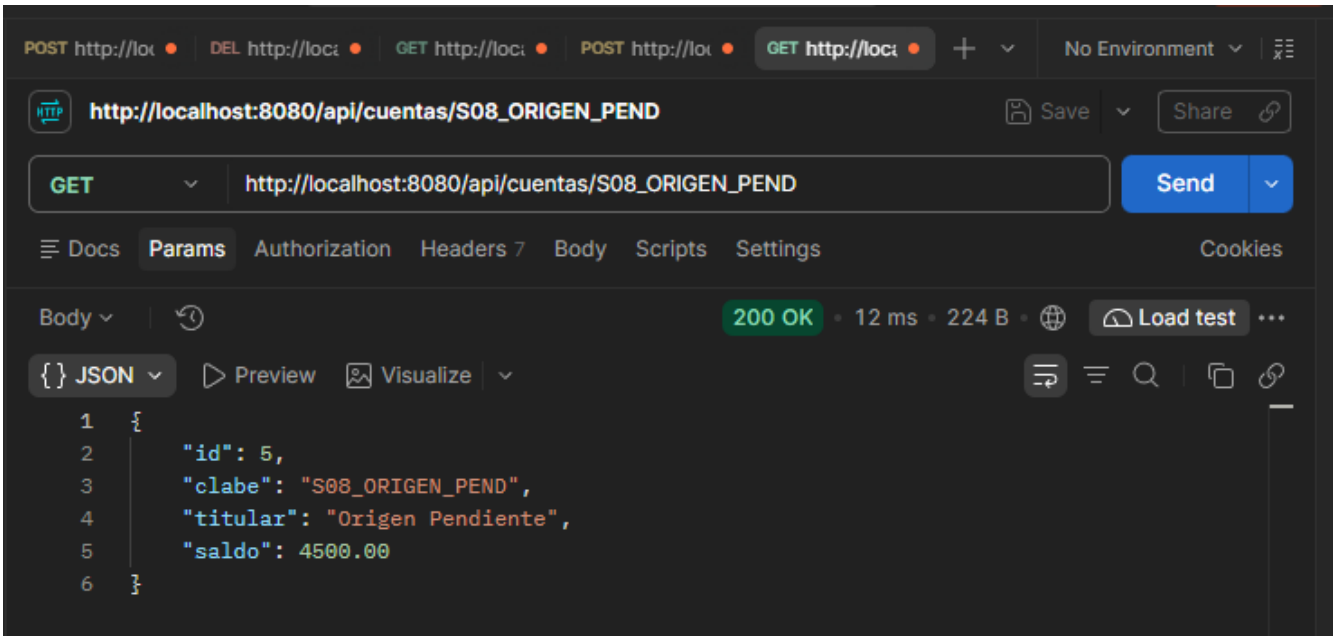


Figure 8: Saldo temporalmente descontado ($4500.00) por fallo total de infraestructura.

Para concluir y dar cierre a las transferencias atascadas en estado `COMPENSACION_PENDIENTE`, se implementaría un proceso automatizado en segundo plano (como un *Background Worker* o *CronJob* en Spring mediante la anotación `@Scheduled`). Este demonio leería periódicamente la base de datos buscando transferencias rezagadas en este estado específico, y reintentaría la llamada de abono (reembolso) hacia

la cuenta origen hasta que el microservicio de cuentas vuelva a estar disponible y responda exitosamente.

¿Por qué la compensación debe ser idempotente?
La compensación debe ser estrictamente idempotente debido a que el hilo de reintentos puede enviar la petición de reembolso en repetidas ocasiones, sobre todo si ocurren fallos de red intermitentes o *timeouts*. Si la operación no estuviera diseñada para ser idempotente, un reintento duplicado que pase por error a través de la red podría abonar los fondos al origen más de una vez, generando dinero de la nada e invalidando la consistencia contable de la institución bancaria.

Parte 2 · Circuit breaker en las llamadas a cuenta

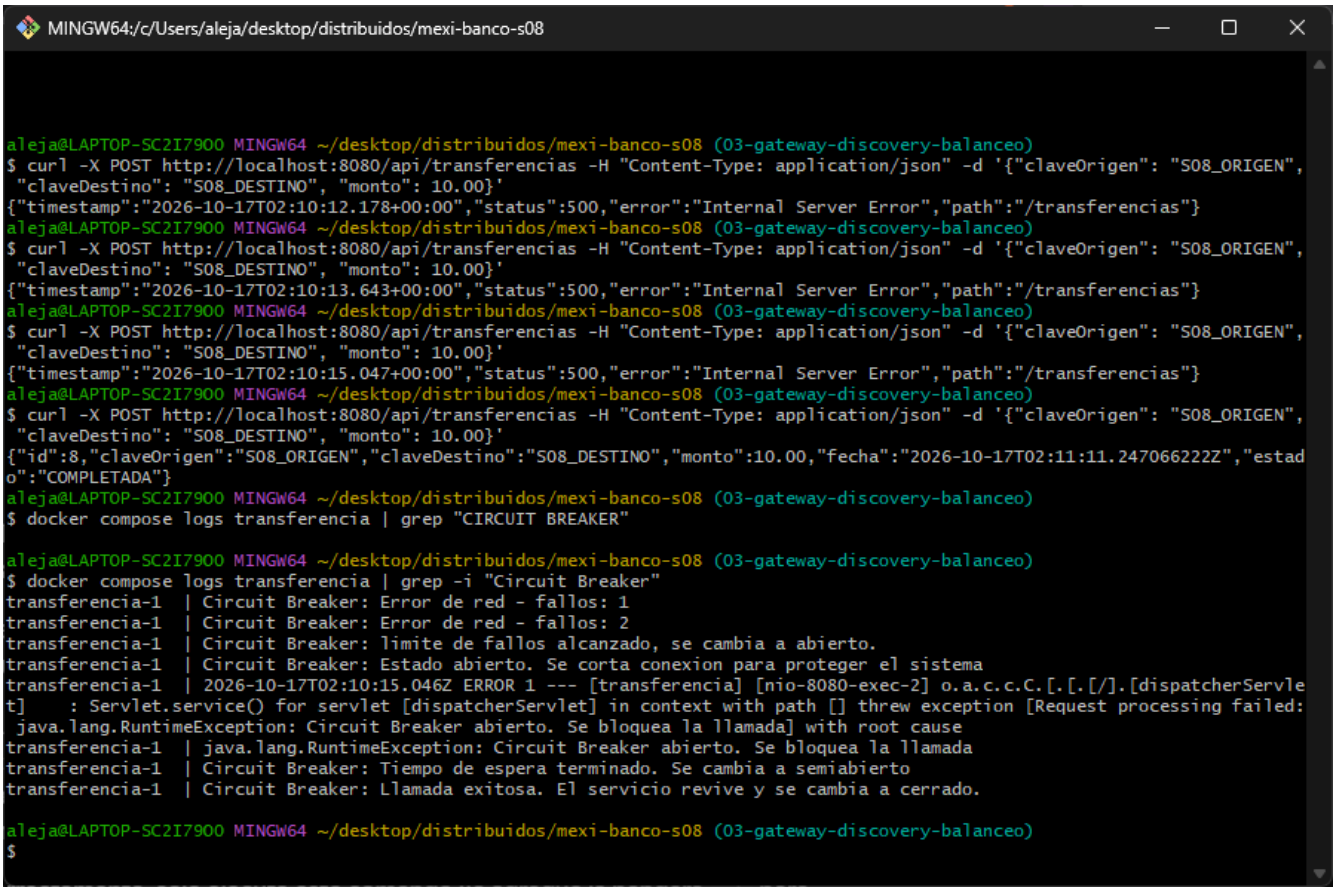


Figure 9: Comportamiento en consola de los estados CERRADO, ABIERTO y SEMIABIERTO.

Para este requerimiento, desarrollé e implementé una solución de *Circuit Breaker* programada de forma manual que protege las llamadas HTTP al microservicio de cuentas.

Como se documenta en los *logs* de la terminal, ante la caída repentina del servicio de cuentas, el circuito registra el fallo. Tras acumular dos fallos consecutivos, cambia de estado CERRADO a ABIERTO. En este estado protector, el sistema corta la conexión de manera proactiva, rechazando peticiones rápidamente sin desgastar recursos ni esperar *timeouts*. Una vez concluido el margen de tiempo configurado (10 segundos), transita a estado SEMIABIERTO para permitir una petición de prueba. Al restablecerse el contenedor

de cuentas y lograr una comunicación exitosa, el circuito asume que el servicio ”revivió” y regresa a su estado CERRADO original, restableciendo el tráfico normal.

Parte 3 · Lo que la saga no te da (Aislamiento)

A diferencia de una transacción monolítica tradicional basada en el modelo ACID, la saga orquestada en un entorno de microservicios carece de una garantía estricta de aislamiento (*Isolation*).

Si un cliente decide consultar el saldo de su cuenta de origen exactamente en el breve intervalo de tiempo en el que se acaba de ejecutar el cargo, pero antes de que se concluya el abono (o se ejecute su debida compensación), el sistema le devolverá el saldo reducido. Este fenómeno evidencia que los estados intermedios e incompletos de la transacción son totalmente visibles para los clientes externos. Aunque la cuenta recuperará sus fondos un milisegundo después mediante el reembolso automático si algo sale mal, el cliente percibirá momentáneamente una reducción de dinero, lo cual es consecuencia directa del modelo de ”consistencia eventual” (*Eventual Consistency*) típico de la arquitectura distribuida.